Base_de_Datos_PL_SQL_Cursores_Apuntes

Propiedades

Base de Datos — PL/SQL: Cursores

Base de Datos

PL/SQL — Cursores explícitos

bases-de-datos × oracle × plsql × cursores × explicit-cursor × rowtype × fetch × notfound × ejercicios ×

+ Añadir propiedad

Base de Datos — PL/SQL: Cursores

Objetivo

Entender qué es un cursor en PL/SQL,por qué se utiliza para procesar varias filas,cómo declararlo,abrirlo, recorrerlo mediante `FETCH`, detectar cuándo ya no quedan filas con `%NOTFOUND`, utilizar `%ROWTYPE` y cerrarlo correctamente.

Cómo usar estos apuntes

Se conservaron los ejercicios de la clase y su estructura, pero cada uno está explicado paso a paso.

Además se incorporaron precisiones de la documentación oficial de Oracle y algunos ejercicios nuevos para practicar de forma progresiva.

Contenido de clase: lo que aparece en el archivo original.
Complemento: explicación técnica y buenas prácticas agregadas a partir de documentación de Oracle.

---

1. ¿Qué es un cursor?

Un **cursor** permite trabajar con el conjunto de fi las que produce una consulta SQL.

Una forma simple de imaginarlo es como un **puntero que recorre las filas del resultado**.

Ejemplo:

```sql
SELECT *
FROM PRODUCTO
ORDER BY nombre_prod;
```

Supongamos que devuelve:

```text
Fila 1 → Monitor DELL
Fila 2 → Mouse HP
Fila 3 → Teclado Microsoft
```

El cursor permite ir obteniendo esas filas una por una:

```text
Cursor
  ↓
Fila 1
  ↓
Fila 2
  ↓
Fila 3
  ↓
No quedan fi las
```

Oracle explica que un cursor explícito se utiliza para procesar de manera controlada el conjunto de resultados de una consulta: se declara, se abre, se hace `FETCH` de las fi las y fi nalmente se cierra.
cite turn326541search4 turn326541search3

---

2. ¿Por qué necesitamos un cursor?

Una consulta SQL puede devolver:

```text
0 fi las
1 fi la
10 fi las
1000 fi las
```

Un `SELECT ... INTO` está pensado para obtener un único resultado de una consulta en una variable o registro.

Cuando necesitamos procesar varias filas dentro de la lógica procedural de PL/SQL,un cursor explícito nos permite recorrer el resultado.

Ejemplo:

```text
SELECT muchos registros
          ↓
        CURSOR
          ↓
procesar uno por uno
```

3. Partes fundamentales de un cursor explícito

La clase trabaja con este esquema:

```SQL
DECLARE

    CURSOR c_productos IS
        SELECT *
        FROM PRODUCTO
        ORDER BY nombre_prod;

    v_producto PRODUCTO%ROWTYPE;

BEGIN

    OPEN c_productos;

    LOOP

        FETCH c_productos  INTO v_producto;

        EXIT WHEN c_productos %NOTFOUND;

        DBMS_OUTPUT.PUT_LINE( v_producto.nombre_prod) ;
```

```
        END LOOP;

        CLOSE c_productos;

END;
/
```

La estructura importante es:

```text
DECLARE
    ↓
Declarar cursor
    ↓
Declarar variable/registro
    ↓
BEGIN
    ↓
OPEN
    ↓
FETCH
    ↓
NOTFOUND?
    ↓
Procesar
    ↓
Repetir
    ↓
CLOSE
END
```

Oracle documenta precisamente las operaciones `OPEN`, `FETCH` y `CLOSE` para los cursores explícitos. citeturn326541search4

---

4. Declarar el cursor

La sintaxis básica es:

```SQL
CURSOR nombre_cursor IS
    SELECT ...
```

Ejemplo:

```SQL
CURSOR c_productos IS
    SELECT *
    FROM PRODUCTO
    ORDER BY nombre_prod;
```

Aquí:

```text
c_productos
```

es el nombre del cursor.

Y:

```SQL
SELECT *
FROM PRODUCTO
ORDER BY nombre_prod;
```

es la consulta asociada al cursor.

Important

Declarar el cursor **no significa que ya esté abierto**.

Solamente estamos definiendo la consulta que utilizará el cursor.

---

5. OPEN

Para utilizar un cursor explícito debemos abrirlo:

```SQL
OPEN c_productos;
```

Según Oracle, `OPEN` prepara el cursor para procesar la consulta y lo posiciona antes de la primera fi la del conjunto de resultados. ⌧cite⌧turn326541search4⌧

Podemos imaginarlo así:

```text
Resultado:

[1]Monitor DELL
[2]Mouse HP
[3]Teclado Microsoft

      ↑
   cursor
```

Después de `OPEN`, el cursor está preparado para comenzar a hacer `FETCH`.

---

6. FETCH

`FETCH` obtiene la siguiente fila del resultado y la coloca en una o más variables.

Ejemplo:

```SQL
FETCH c_productos INTO v_producto;
```

Aquí:

```text
c_productos
      ↓
obtiene siguiente fi la
      ↓
v_producto
```

Si la primera fila es:

```text
1 | Monitor DELL
```

después del `FETCH` tenemos:

```text
v_producto.id_prod = 1
v_producto.nombre_prod = 'Monitor DELL'
```

Oracle indica que `FETCH` recupera la fi la actual, almacena sus valores en variables o en un registro y avanza el cursor hacia la siguiente fi la. cite turn326541search4

---

7. `%ROWTYPE`

Esta es una de las partes más importantes de la clase.

En vez de declarar cada variable por separado:

```SQL
v_id_producto NUMBER;
v_nombre_producto VARCHAR2(50);
```

podemos utilizar:

```SQL
v_producto PRODUCTO%ROWTYPE;
```

Esto crea una variable tipo **record** con la estructura de una fi la de la tabla `PRODUCTO`.

Si la tabla tiene:

```text
id_prod
nombre_prod
```

entonces:

```SQL
v_producto PRODUCTO%ROWTYPE;
```

nos permite usar:

```SQL
v_producto.id_prod
v_producto.nombre_prod
```

Oracle documenta `%ROWTYPE` como una forma de declarar registros que contienen los mismos campos que una fi la de una tabla, vista o cursor. También permite utilizar registros en `FETCH`. ⌧cite⌧turn326541search4⌧

---

8. ¿Por qué `%ROWTYPE` es útil?

Sin `%ROWTYPE`:

```SQL
v_id NUMBER;
v_nombre VARCHAR2(50);
```

Con `%ROWTYPE`:

```SQL
v_producto PRODUCTO%ROWTYPE;
```

Esto hace el código más cómodo cuando necesitamos copiar **todas las columnas de una fila**.

La correspondencia es:

```text
Tabla PRODUCTO
        ↓
PRODUCTO%ROWTYPE
        ↓
v_producto
```

9. LOOP

Después de abrir el cursor utilizamos un ciclo para ir obteniendo las filas:

```sql
LOOP

    FETCH c_productos  INTO  v_producto;

    EXIT WHEN c_productos %NOTFOUND;

    ...

END LOOP;
```

La idea es:

```text
FETCH fi la 1
↓
procesar

FETCH fi la 2
↓
procesar

FETCH fi la 3
↓
procesar

FETCH
↓
no quedan fi las
↓
EXIT
```

10. %NOTFOUND

`%NOTFOUND` indica si el último `FETCH` **no pudo devolver una fila**.

Oracle especifica:

> Después de `OPEN`, antes del primer `FETCH`: `%NOTFOUND` es `NULL`.
> Si el último `FETCH` devolvió una fila: `%NOTFOUND` es `FALSE`.
> Si el último `FETCH` no devolvió una fila: `%NOTFOUND` es `TRUE`. ⌧cite⌧turn326541search2⌧turn326541search4⌧

Por eso usamos:

```sql
EXIT WHEN c_productos%NOTFOUND;
```

Significa:

> "Sal del ciclo cuando el cursor ya no tenga otra fi la para entregar."

11. Orden correcto: FETCH → NOTFOUND → procesar

Esta es una idea fundamental.

La secuencia recomendable es:

```sql
FETCH c_productos INTO v_producto;

EXIT WHEN c_productos%NOTFOUND;

DBMS_OUTPUT.PUT_LINE(v_producto.nombre_prod);
```

¿Por qué?

Porque primero intentamos obtener una fi la.

Luego preguntamos:

```SQL
¿El FETCH encontró una fila      ?
```

Si la respuesta es no:

```SQL
EXIT
```

Solo si encontró una fila la procesamos.

Diagrama

```text
FETCH
 ↓
%NOTFOUND?
 ├── Sí → EXIT
 └── No → procesar fila
              ↓
           repetir
```

Oracle muestra este patrón explícitamente en su documentación de cursores.
⌧cite⌧turn326541search2⌧turn326541search8⌧

---

12. CLOSE

Cuando terminamos de utilizar el cursor:

```sql
CLOSE c_productos;
```

Oracle indica que `CLOSE` libera los recursos asociados al cursor. Después de cerrarlo ya no se puede hacer `FETCH` sobre su conjunto de resultados y volver a utilizar sus atributos puede producir `INVALID_CURSOR`.
⌧cite⌧turn326541search4⌧

Por eso:

```text
OPEN
 ↓
FETCH
 ↓
...
 ↓
CLOSE
```

es un ciclo completo.

---

13. Las cuatro etapas para memorizar

> 🔥 **Fórmula para examen**

```text
OPEN
 ↓
FETCH
 ↓
CHECK %NOTFOUND
 ↓
CLOSE
```

Pero en un ciclo:

```text
OPEN
 ↓
LOOP
  ↓
FETCH
  ↓
EXIT WHEN %NOTFOUND
  ↓
PROCESAR
  ↓
REPETIR
 ↓
CLOSE
```

Frase para memorizar

Abro → saco una fila → compruebo → proceso → repito → cierro.

14. Atributos de un cursor explícito

Los atributos principales son:

Atributo	Significado
`%ISOPEN`	Indica si el cursor está abierto
`%FOUND`	Indica si el último `FETCH` encontró una fila
`%NOTFOUND`	Indica si el último `FETCH` no encontró una fila
`%ROWCOUNT`	Indica cuántas filas se han obtenido hasta el momento

Oracle documenta estos cuatro atributos para los cursores explícitos.
cite turn326541search1 turn326541search7

---

15. `%FOUND`

Es prácticamente lo contrario de `%NOTFOUND`.

Después de un `FETCH` exitoso:

```sql
c_productos%FOUND
```

es:

```text
TRUE
```

Si no se obtuvo una fila:

```text
FALSE
```

Ejemplo:

```SQL
FETCH c_productos INTO v_producto;

IF c_productos %FOUND THEN
    DBMS_OUTPUT.PUT_LINE( 'Fila encontrada' );
END IF ;
```

16. %ISOPEN

Indica si el cursor está abierto.

```SQL
c_productos %ISOPEN
```

Puede devolver:

```text
TRUE
FALSE
```

Ejemplo:

```SQL
IF NOT c_productos %ISOPEN THEN
    OPEN c_productos;
END IF ;
```

Oracle especifica que intentar abrir un cursor que ya está abierto provoca `CURSOR_ALREADY_OPEN`.
citeturn326541search7

---

17. `%ROWCOUNT`

Indica cuántas filas se han recuperado mediante `FETCH`.

Ejemplo:

```SQL
FETCH c_productos INTO v_producto;
```

Después del primer `FETCH` exitoso:

```text
%ROWCOUNT = 1
```

Después del segundo:

```text
%ROWCOUNT = 2
```

Después del tercero:

```text
%ROWCOUNT = 3
```

Esto es útil para saber cuántas filas se han procesado.

---

18. Ejercicio 1 — Productos

Código original de clase

```sql
CREATE TABLE PRODUCTO(

    id_prod NUMBER PRIMARY KEY,

    nombre_prod VARCHAR2(50) NOT NULL

);

INSERT INTO PRODUCTO VALUES (1, 'Monitor DELL' );
INSERT INTO PRODUCTO VALUES (2, 'Mouse HP' );
INSERT INTO PRODUCTO VALUES (3, 'Teclado Microsoft' );

DECLARE

    CURSOR c_productos IS
        SELECT *
        FROM PRODUCTO
        ORDER BY nombre_prod;

    v_producto PRODUCTO%ROWTYPE;

BEGIN

    OPEN c_productos;

    LOOP

        FETCH c_productos INTO v_producto;

        EXIT WHEN c_productos%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Informacion del producto con código ' ||
            v_producto.id_prod
```

```
        );

        DBMS_OUTPUT.PUT_LINE(
            'Nombre del producto con código '  ||
            v_producto.nombre_prod
        );

    END LOOP;

    CLOSE c_productos;

END;
/
```

La estructura original de este ejercicio aparece en la transcripción/material de la clase. filecite turn3file0 L5-L67

---

19. Análisis del ejercicio 1

Paso 1 — Crear tabla

```sql
CREATE TABLE PRODUCTO(
    id_prod NUMBER PRIMARY KEY,
    nombre_prod VARCHAR2(50) NOT NULL
);
```

La tabla tiene dos columnas:

```text
id_prod
nombre_prod
```

Paso 2 — Insertar datos

```SQL
INSERT INTO PRODUCTO VALUES (1, 'Monitor DELL' );
INSERT INTO PRODUCTO VALUES (2, 'Mouse HP');
INSERT INTO PRODUCTO VALUES (3, 'Teclado Microsoft' );
```

Tenemos:

```text
1 → Monitor DELL
2 → Mouse HP
3 → Teclado Microsoft
```

Paso 3 — Declarar el cursor

```SQL
CURSOR c_productos IS
    SELECT *
    FROM PRODUCTO
    ORDER BY nombre_prod;
```

El cursor queda asociado a esta consulta.

El resultado se ordena por:

```SQL
nombre_prod
```

Por lo tanto, el orden no depende del `id_prod`.

Conceptualmente podríamos obtener:

```text
Monitor DELL
Mouse HP
Teclado Microsoft
```

Paso 4 — Declarar el registro

```SQL
v_producto PRODUCTO%ROWTYPE;
```

Creamos un registro con la estructura de una fi la de `PRODUCTO`.

Podemos acceder a:

```SQL
v_producto.id_prod
v_producto.nombre_prod
```

Paso 5 — Abrir el cursor

```SQL
OPEN c_productos;
```

Ahora la consulta está preparada y el cursor está antes de la primera fila.

Paso 6 — FETCH

```sql
FETCH c_productos INTO v_producto;
```

El cursor toma una fila y la copia a:

```text
v_producto
```

Primera iteración:

```text
v_producto.id_prod = 1
v_producto.nombre_prod = Monitor DELL
```

Paso 7 — Comprobar `%NOTFOUND`

```sql
EXIT WHEN c_productos %NOTFOUND;
```

Si la fila existía:

```text
NOTFOUND = FALSE
```

El programa continúa.

Cuando el `FETCH` intenta obtener una fila que no existe:

```text
NOTFOUND = TRUE
```

y el ciclo termina.

Paso 8 — Mostrar la información

```sql
DBMS_OUTPUT.PUT_LINE( ... );
```

Utilizamos los campos del registro:

```sql
v_producto.id_prod
v_producto.nombre_prod
```

Paso 9 — Cerrar

```sql
CLOSE c_productos;
```

Terminamos de trabajar con el cursor.

---

20. Flujo completo del ejercicio 1

```text
OPEN
 ↓
FETCH → Producto 1
 ↓
NOTFOUND = FALSE
 ↓
IMPRIMIR
 ↓
FETCH → Producto 2
 ↓
NOTFOUND = FALSE
 ↓
IMPRIMIR
 ↓
FETCH → Producto 3
 ↓
NOTFOUND = FALSE
 ↓
IMPRIMIR
 ↓
FETCH → no existe fila
 ↓
NOTFOUND = TRUE
 ↓
EXIT
 ↓
CLOSE
```

Este diagrama es probablemente la mejor forma de entender el ejercicio.

21. Ejercicio 2 — Clientes

Enunciado de la clase

> Generar un bloque anónimo con cursor que muestre los registros de la tabla cliente.

Datos solicitados:

- RUT.
- Nombre.
- Sueldo base.

La actividad aparece en el material original de la clase. ⌧fi lecite⌧turn3fi le0⌧L69-L79⌧

---

22. Preparación de la tabla

```SQL
CREATE TABLE CLIENTES(

    rut VARCHAR2(10)  PRIMARY KEY NOT NULL,

    nombre_cliente VARCHAR2(50)  NOT NULL,

    sueldobase NUMBER(10)  NOT NULL

);
```

Tenemos tres columnas:

```text
rut
nombre_cliente
sueldobase
```

23. Datos insertados

```SQL
INSERT INTO CLIENTES VALUES
('18977594-6' , 'Juan Perez' , 500000);

INSERT INTO CLIENTES VALUES
('18977585-5' , 'Guidoberto Villaroel' , 3000000);

INSERT INTO CLIENTES VALUES
('9787835-5' , 'David Almonacid' , 10000);
```

24. Cursor del ejercicio 2

La consulta usada en clase:

```SQL
CURSOR c_clientes IS
    SELECT *
    FROM CLIENTES
    ORDER BY sueldobase DESC;
```

El `DESC` significa **descendente**.

Por lo tanto, los sueldos aparecen de mayor a menor:

```text
3.000.000
  500.000
   10.000
```

25. Registro `%ROWTYPE`

```SQL
v_cliente CLIENTES %ROWTYPE;
```

Ahora podemos utilizar:

```SQL
v_cliente .rut
v_cliente .nombre_cliente
v_cliente .sueldobase
```

Esto evita tener que declarar tres variables manualmente.

26. Una observación importante del ejercicio original

El código de la transcripción tiene:

```SQL
FETCH c_clientes INTO v_cliente ;

DBMS_OUTPUT.PUT_LINE( ... v_cliente ... );

EXIT WHEN c_clientes %NOTFOUND;
```

Es decir:

```text
FETCH
 ↓
IMPRIMIR
 ↓
NOTFOUND
```

Esto **no es el orden recomendable**.

La forma segura es:

```sql
FETCH c_clientes INTO v_cliente;
EXIT WHEN c_clientes%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(...);
```

¿Por qué?

Porque cuando el `FETCH` ya no devuelve una fila, `%NOTFOUND` se vuelve `TRUE`. Ese es justamente el momento en que debemos salir del ciclo antes de procesar la variable. Oracle utiliza este patrón en su documentación oficial. cite turn326541search2 turn326541search8

---

27. Versión recomendada del ejercicio 2

```sql
DECLARE
    CURSOR c_clientes IS
        SELECT *
        FROM CLIENTES
        ORDER BY sueldobase DESC;
    v_cliente CLIENTES%ROWTYPE;
BEGIN
    OPEN c_clientes;
    LOOP
        FETCH c_clientes INTO v_cliente;
        EXIT WHEN c_clientes%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(
```

```
            'Rut: ' || v_cliente.rut
        );

        DBMS_OUTPUT.PUT_LINE(
            'Nombre: ' || v_cliente.nombre_cliente
        );

        DBMS_OUTPUT.PUT_LINE(
            'Sueldo base: ' || v_cliente.sueldobase
        );

        DBMS_OUTPUT.PUT_LINE('-------------------');

    END LOOP;

    CLOSE c_clientes;

END;
/
```

28. Resultado conceptual

```text
Rut: 18977585-5
Nombre: Guidoberto Villaroel
Sueldo base: 3000000
-------------------

Rut: 18977594-6
Nombre: Juan Perez
Sueldo base: 500000
-------------------

Rut: 9787835-5
Nombre: David Almonacid
Sueldo base: 10000
-------------------
```

29. Cursor vs SELECT INTO

Es importante distinguirlos.

SELECT INTO

Se utiliza normalmente cuando esperamos un único resultado.

Ejemplo:

```SQL
SELECT nombre_cliente
INTO v_nombre
FROM CLIENTES
WHERE rut = '18977594-6' ;
```

La consulta pretende entregar una fi la.

Cursor

Se utiliza cuando queremos trabajar con múltiples filas:

```SQL
CURSOR c_clientes IS
    SELECT *
    FROM CLIENTES;
```

Y posteriormente:

```SQL
FETCH
```

para ir obteniendo una por una.

Regla mental

```text
1 fila → SELECT INTO

Muchas filas → cursor
```

Es una simplificación didáctica, pero útil para esta etapa.

---

30. Cursor explícito vs cursor implícito

La clase incluye **cursor implícito** como materia de prueba.

Cursor implícito

Oracle administra automáticamente el cursor asociado a una sentencia SQL.

Ejemplo conceptual:

```sql
UPDATE CLIENTES
SET sueldobase = sueldobase + 10000
WHERE rut = '18977594-6' ;
```

Oracle administra el cursor internamente y podemos consultar atributos mediante:

```sql
SQL%ROWCOUNT
SQL%FOUND
SQL%NOTFOUND
```

Oracle distingue el cursor implícito `SQL` de los cursores explícitos nombrados por el programador. ￼cite￼turn326541search0￼turn326541search9￼

Cursor explícito

Nosotros declaramos el cursor:

```SQL
CURSOR c_clientes  IS
    SELECT ...
```

y controlamos:

```text
OPEN
FETCH
CLOSE
```

---

31. Comparación rápida

Cursor implícito	Cursor explícito
Oracle lo administra	Nosotros lo declaramos y controlamos
Se usa automáticamente con SQL	Se defi ne con `CURSOR`
Atributos como `SQL%ROWCOUNT`	Atributos como `c_clientes%ROWCOUNT`
No hacemos `OPEN/FETCH/CLOSE` manualmente	Sí hacemos `OPEN/FETCH/CLOSE`
Útil para operaciones SQL directas	Útil para procesar resultados fila por fila

32. `FETCH` y `%ROWTYPE`

Este patrón debes aprenderlo:

```sql
DECLARE

    CURSOR c_clientes IS
        SELECT *
        FROM CLIENTES;

    v_cliente CLIENTES %ROWTYPE;

BEGIN

    OPEN c_clientes ;

    LOOP

        FETCH c_clientes INTO v_cliente ;

        EXIT WHEN c_clientes %NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            v_cliente .nombre_cliente
        );

    END LOOP;

    CLOSE c_clientes ;

END;
/
```

Hay una relación directa:

```text
SELECT *
    ↓
CURSOR
    ↓
FETCH
    ↓
%ROWTYPE
    ↓
v_cliente.columna
```

---

33. Cursores parametrizados — introducción

La materia de la prueba también incluye **cursores parametrizados**.

Un cursor parametrizado permite entregar valores al cursor al momento de abrirlo.

Sintaxis:

```sql
CURSOR c_clientes ( p_sueldo NUMBER) IS
    SELECT *
    FROM CLIENTES
    WHERE sueldobase >= p_sueldo;
```

Luego:

```sql
OPEN c_clientes (500000);
```

Ahora el cursor devolverá solamente los clientes cuyo sueldo sea mayor o igual a `500000`.

Oracle documenta los cursores explícitos que aceptan parámetros como una forma de reutilizar el mismo cursor con distintos valores. ⌧cite⌧turn326541search4⌧

---

34. Ejemplo de cursor parametrizado

```sql
DECLARE

    CURSOR c_clientes ( p_sueldo NUMBER) IS
        SELECT *
        FROM CLIENTES
        WHERE sueldobase >= p_sueldo
        ORDER BY sueldobase DESC;

    v_cliente CLIENTES %ROWTYPE;

BEGIN

    OPEN c_clientes (500000);

    LOOP

        FETCH c_clientes INTO v_cliente ;

        EXIT WHEN c_clientes %NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            v_cliente .nombre_cliente ||
            '-' ||
            v_cliente .sueldobase
        );

    END LOOP;

    CLOSE c_clientes ;

END;
/
```

La consulta queda parametrizada.

```text
OPEN c_clientes(500000)
            ↓
p_sueldo = 500000
            ↓
SELECT ...
WHERE sueldobase >= 500000
```

35. Cursores + IF

Los cursores se pueden combinar con condiciones.

Ejemplo:

```SQL
IF v_cliente    .sueldobase   >= 1000000 THEN

    DBMS_OUTPUT.PUT_LINE(
        'Sueldo alto'
    );

END IF ;
```

Así podemos procesar cada fila de manera diferente.

---

36. Cursores + ciclos

Esto es muy importante porque conecta varias materias.

```text
CURSOR
  +
FETCH
  +
LOOP
  +
IF
```

Ejemplo:

```SQL
LOOP

    FETCH c_clientes  INTO v_cliente ;

    EXIT WHEN c_clientes %NOTFOUND;

    IF v_cliente .sueldobase >= 1000000 THEN
        DBMS_OUTPUT.PUT_LINE( 'Sueldo alto' );
    ELSE
        DBMS_OUTPUT.PUT_LINE( 'Sueldo normal' );
    END IF ;

END LOOP;
```

37. Cursores + ciclos anidados

Como la prueba también considera ciclos anidados, eventualmente puedes encontrar algo como:

```SQL
LOOP

    FETCH c_clientes   INTO v_cliente  ;

    EXIT WHEN c_clientes  %NOTFOUND;

    FOR i  IN 1.. 3
    LOOP
        ...
    END LOOP;

END LOOP;
```

La idea es tener un ciclo dentro de otro.

Por ahora, lo fundamental es dominar primero:

```text
cursor
+
OPEN
+
FETCH
+
NOTFOUND
+
CLOSE
```

38. Errores frecuentes

1. Olvidar `OPEN`

Incorrecto:

```sql
FETCH c_clientes  INTO v_cliente ;
```

si nunca hicimos:

```sql
OPEN c_clientes ;
```

2. Olvidar `CLOSE`

Siempre que controles manualmente un cursor explícito, acostúmbrate a terminar con:

```sql
CLOSE c_clientes ;
```

3. Colocar `%NOTFOUND` después de imprimir

Evita:

```sql
FETCH ...
PRINT ...
EXIT WHEN %NOTFOUND;
```

Prefiere:

```SQL
FETCH ...

EXIT WHEN %NOTFOUND;

PRINT ...
```

4. Confundir cursor con variable

Esto:

```SQL
c_clientes
```

es el cursor.

Esto:

```SQL
v_cliente
```

es el registro donde guardamos la fi la obtenida.

```text
CURSOR → obtiene
VARIABLE/RECORD → recibe
```

5. Confundir `%ROWTYPE` con `%TYPE`

`%TYPE`

Hereda el tipo de **una columna**.

```sql
v_nombre CLIENTES.nombre_cliente%TYPE;
```

`%ROWTYPE`

Representa la estructura de una **fila completa**.

```sql
v_cliente CLIENTES%ROWTYPE;
```

Memoria rápida

```text
%TYPE → una columna
%ROWTYPE → una fila
```

39. Diagrama mental definitivo

```text
            CURSOR
              |
              ↓
        SELECT asociado
              |
             OPEN
              |
              ↓
            FETCH
              |
              ↓
       %NOTFOUND ?
         / \
       SÍ NO
       ↓ ↓
  EXIT %ROWTYPE
                ↓
           PROCESAR
                ↓
           repetir
                |
                └──→ FETCH

                    al terminar
                          ↓
                       CLOSE
```

40. Ejercicio de práctica 1 — Empleados

Crea una tabla:

```SQL
CREATE TABLE EMPLEADOS(
    id_empleado NUMBER PRIMARY KEY,
    nombre VARCHAR2(50),
    sueldo NUMBER
);
```

Inserta cinco empleados.

Luego crea un cursor que:

- Recorra todos los empleados.
- Muestre nombre y sueldo.
- Use `%ROWTYPE`.
- Utilice `OPEN`, `FETCH`, `%NOTFOUND` y `CLOSE`.

Objetivo

Repetir exactamente el patrón de los ejercicios de clase.

41. Ejercicio de práctica 2 — Ordenar empleados

Utiliza la tabla anterior.

Crea un cursor que muestre los empleados:

```text
del mayor sueldo al menor
```

Pista

Utiliza:

```sql
ORDER BY sueldo DESC
```

Objetivo

Practicar:

```text
CURSOR
+
SELECT
+
ORDER BY
+
FETCH
```

42. Ejercicio de práctica 3 — Sueldo alto

Recorre los empleados.

Si el sueldo es mayor o igual a:

```text
1.000.000
```

muestra:

```text
Empleado con sueldo alto
```

De lo contrario:

```text
Empleado con sueldo normal
```

Objetivo

Practicar:

```text
CURSOR + IF
```

---

43. Ejercicio de práctica 4 — Contador

Recorre todos los empleados y cuenta cuántos existen.

Debes utilizar una variable:

```SQL
v_contador NUMBER := 0;
```

Dentro del ciclo:

```SQL
v_contador := v_contador + 1;
```

Al final muestra:

```text
Cantidad de empleados: X
```

Objetivo

Practicar:

```text
FETCH
+
COUNT lógico con variable
+
ciclo
```

---

44. Ejercicio de práctica 5 — Suma de sueldos

Recorre todos los empleados y calcula cuánto dinero representan todos los sueldos juntos.

Ejemplo:

```text
500000
700000
900000
1200000
1500000
```

Resultado:

```text
Suma total: 4.800.000
```

Pista

```sql
v_total := v_total + v_empleado.sueldo ;
```

---

45. Ejercicio de práctica 6 — Promedio

Usando el mismo cursor:

- Suma todos los sueldos.
- Cuenta empleados.
- Calcula el promedio.

```text
Promedio = suma / cantidad
```

Objetivo

Combinar:

```text
CURSOR
+
%ROWTYPE
+
FOR/LOOP
+
variables acumuladoras
```

46. Ejercicio de práctica 7 — Buscar empleado

Crea un cursor que recorra los empleados y busque:

```text
"Juan"
```

Si lo encuentra:

```text
Empleado encontrado
```

Si termina el cursor y no lo encuentra:

```text
Empleado no encontrado
```

Objetivo

Aprender a diferenciar:

```text
encontré una fila concreta
```

de:

```text
terminé de recorrer todas las filas
```

47. Ejercicio de práctica 8 — Cursor parametrizado

Crea:

```SQL
CURSOR c_empleados( p_sueldo NUMBER)  IS
    SELECT *
    FROM EMPLEADOS
    WHERE sueldo  >= p_sueldo;
```

Luego ejecútalo con:

```SQL
OPEN c_empleados( 1000000) ;
```

Muestra solamente los empleados que cumplen la condición.

Objetivo

Practicar **cursor parametrizado**.

48. Ejercicio de práctica 9 — Cursor + dos condiciones

Recorre los empleados y clasifícalos:

```text
>= 2.000.000 → Ejecutivo
>= 1.000.000 → Profesional
< 1.000.000 → Operacional
```

Utiliza:

```SQL
IF
ELSIF
ELSE
```

Objetivo

Practicar:

```text
CURSOR + IF/ELSIF/ELSE
```

49. Desafío final

Informe de empleados

Crea un bloque PL/SQL que utilice un cursor para generar un pequeño informe de empleados.

El programa debe:

Mostrar todos los empleados.
Mostrar su sueldo.
Indicar si su sueldo es alto o normal.
Contar cuántos empleados existen.
Calcular el sueldo total.
Calcular el sueldo promedio.
Mostrar el empleado con el sueldo más alto.
Mostrar el empleado con el sueldo más bajo.

Debes utilizar

`CURSOR`.
`OPEN`.
`FETCH`.
`%NOTFOUND`.
`%ROWTYPE`.
`LOOP`.
`IF`.
Variables acumuladoras.

No utilizar todavía

`BULK COLLECT`.
`FORALL`.
Funciones avanzadas de colecciones.

La idea es resolverlo utilizando exactamente lo aprendido en esta clase.

---

50. Cómo resolver el desafío

Divide el problema.

Variables

Necesitarás aproximadamente:

```text
v_empleado
v_contador
v_total
v_promedio
v_mayor
v_menor
```

Algoritmo

```text
OPEN cursor
    ↓
Inicializar variables
    ↓
FETCH
    ↓
¿NOTFOUND?
    ├── Sí → terminar
    └── No
          ↓
        contar
          ↓
        sumar
          ↓
        comparar mayor
          ↓
        comparar menor
          ↓
        mostrar
          ↓
        volver a FETCH
    ↓
CLOSE
    ↓
calcular promedio
    ↓
mostrar resumen
```

51. Sintaxis que debes memorizar

Cursor básico

```SQL
CURSOR c_nombre IS
    SELECT ...
```

Variable de fila

```SQL
v_dato tabla %ROWTYPE;
```

Abrir

```SQL
OPEN c_nombre;
```

Obtener fila

```SQL
FETCH c_nombre INTO v_dato;
```

Comprobar fin

```SQL
EXIT WHEN c_nombre%NOTFOUND;
```

Cerrar

```SQL
CLOSE c_nombre;
```

Recorrer

```sql
LOOP

    FETCH c_nombre INTO v_dato;

    EXIT WHEN c_nombre%NOTFOUND;

    ...

END LOOP;
```

52. Plantilla base para copiar y adaptar

```sql
DECLARE

    CURSOR c_datos IS
        SELECT *
        FROM TABLA;

    v_dato TABLA%ROWTYPE;

BEGIN

    OPEN c_datos;

    LOOP

        FETCH c_datos INTO v_dato;

        EXIT WHEN c_datos%NOTFOUND;

        -- Procesar fila
        DBMS_OUTPUT.PUT_LINE(v_dato.columna);

    END LOOP;

    CLOSE c_datos;

END;
```

```
/
```

🔥 **Esta plantilla es la que conviene aprender a escribir de memoria.**

---

53. Resumen de la clase

Un **cursor explícito** permite recorrer el resultado de una consulta fi la por fi la.

La secuencia fundamental es:

```text
DECLARE
   ↓
CURSOR
   ↓
%ROWTYPE
   ↓
OPEN
   ↓
LOOP
   ↓
FETCH
   ↓
%NOTFOUND
   ↓
PROCESAR
   ↓
LOOP
   ↓
CLOSE
```

Conceptos esenciales

CURSOR

> Puntero/controlador para recorrer las fi las de un resultado.

OPEN

> Abre el cursor y prepara el resultado.

FETCH

Obtiene la siguiente fi la y la copia en una variable o registro.

%ROWTYPE

Permite declarar una variable con la estructura de una fi la.

%NOTFOUND

Indica que el último `FETCH` no encontró una fila.

%ROWCOUNT

Indica cuántas filas han sido obtenidas.

%FOUND

Indica si el último `FETCH` encontró una fila.

%ISOPEN

Indica si el cursor está abierto.

CLOSE

Cierra el cursor y libera sus recursos.

54. Diferencias clave para la prueba

Concepto	Qué recordar
`%TYPE`	Tipo de una columna/variable
`%ROWTYPE`	Estructura de una fi la
`COUNT` de VARRAY	Elementos actuales
`LIMIT` de VARRAY	Capacidad máxima
`CURSOR`	Recorre resultados
`OPEN`	Abre cursor
`FETCH`	Obtiene una fi la
`%NOTFOUND`	No quedan fi las
`%FOUND`	Se obtuvo una fi la
`%ROWCOUNT`	Filas obtenidas
`%ISOPEN`	Cursor abierto/cerrado
`CLOSE`	Cierra cursor
Cursor implícito	Oracle lo administra
Cursor explícito	Lo controlamos nosotros
Cursor parametrizado	Recibe parámetros al abrirse

55. Autoevaluación

Intenta contestar sin mirar los apuntes.

- ¿Qué es un cursor?
- ¿Por qué utilizar un cursor en vez de procesar directamente una consulta?
- ¿Qué diferencia existe entre cursor implícito y explícito?
- ¿Qué hace `OPEN`?
- ¿Qué hace `FETCH`?
- ¿Qué hace `CLOSE`?
- ¿Qué significa `%NOTFOUND`?
- ¿Qué significa `%FOUND`?
- ¿Qué significa `%ROWCOUNT`?
- ¿Qué significa `%ISOPEN`?
- ¿Por qué utilizamos `%ROWTYPE`?
- ¿Qué diferencia hay entre `%TYPE` y `%ROWTYPE`?

¿Por qué debemos comprobar `%NOTFOUND` después de `FETCH` y antes de procesar la fi la?
¿Qué ocurre si intentamos abrir un cursor que ya está abierto?
¿Qué ocurre si hacemos `FETCH` cuando el cursor está cerrado?
¿Qué es un cursor parametrizado?
¿Para qué sirven los parámetros de un cursor?
¿Cómo combinarías un cursor con un `IF`?
¿Cómo contarías las filas procesadas?
¿Cómo calcularías el promedio de una columna usando un cursor?

---

56. Mini prueba

1

¿Qué instrucción se utiliza para abrir un cursor explícito?

Respuesta

```sql
OPEN nombre_cursor;
```

2

¿Qué instrucción obtiene la siguiente fila?

Respuesta

```sql
FETCH nombre_cursor INTO variable;
```

3

¿Qué atributo se utiliza normalmente para terminar un loop de un cursor?

Respuesta

```sql %NOTFOUND ``` Por ejemplo: ```sql EXIT WHEN c_clientes%NOTFOUND; ```

4

¿Qué significa `CLIENTES%ROWTYPE`?

Respuesta

Una variable con la estructura de una fila de la tabla `CLIENTES`.

5

¿Qué diferencia hay entre `%TYPE` y `%ROWTYPE`?

Respuesta

`%TYPE` permite heredar el tipo de una columna o variable. `%ROWTYPE` permite representar la estructura de una fila completa.

6

¿Por qué es recomendable poner `%NOTFOUND` antes de procesar la variable?

Respuesta

Porque primero debemos verificar que el `FETCH` realmente obtuvo una fila. Cuando el `FETCH` no encuentra otra fila, `%NOTFOUND` se vuelve `TRUE` y debemos salir del ciclo.

7

¿Qué hace `%ROWCOUNT`?

Respuesta
```

Indica cuántas filas ha recuperado el cursor mediante `FETCH`.

8

¿Qué hace un cursor parametrizado?

Respuesta

Permite recibir valores al abrir el cursor y utilizarlos en la consulta.

---

57. Fuentes

Documentación oficial de Oracle

Oracle Database — Cursors Overview
https://docs.oracle.com/en/database/oracle/oracle-database/26/lnpls/cursors-overview.html

Oracle Database — Using Records and Cursors
https://docs.oracle.com/en/database/oracle/oracle-database/26/tdddg/using-records-and-cursors.html

Oracle Database — Cursor Attributes
https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/cursor-attributes.html

Oracle Database — PL/SQL Language Reference
https://docs.oracle.com/en/database/oracle/oracle-database/

> **Note**
>
> La fuente principal de los ejercicios y del orden de contenidos es el material de la clase. fi lecite turn3fi le0L1-L67 fi lecite turn3fi le0L69-L135

Las definiciones y precisiones técnicas sobre `OPEN`, `FETCH`, `CLOSE`, `%NOTFOUND`, `%FOUND`, `%ISOPEN`, `%ROWCOUNT`, `%ROWTYPE` y cursores parametrizados fueron contrastadas con la documentación de Oracle. cite turn326541search4turn326541search2turn326541search1

---

58. La frase que debes recordar

> **Important**
>
> **CURSOR = OPEN → FETCH → NOTFOUND → PROCESAR → REPETIR → CLOSE**

Y para `%ROWTYPE`:

```
%TYPE = una columna
%ROWTYPE = una fila
```